

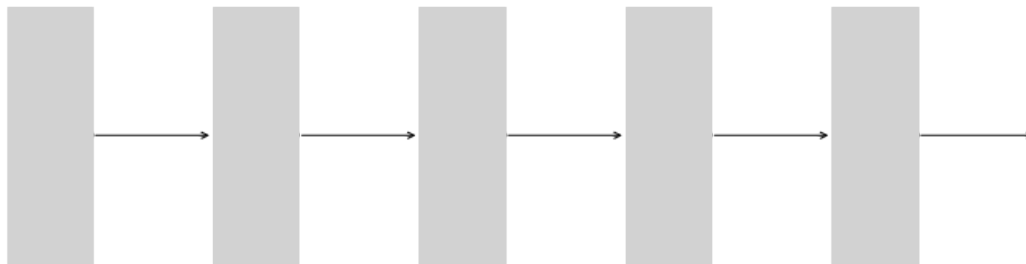
Neural Network

A Neural Network (Sequential architecture) is composed of

- sequence of Layers
 - layer $\ll$ transforms its input $\mathbf{y}_{(\ll-1)}$ to output $\mathbf{y}_{\ll p}$
 - through a transformation: operation parameterized by weights $\mathbf{W}_{\ll p}$

In [6]: `fig_tf_seq`

Out[6]:



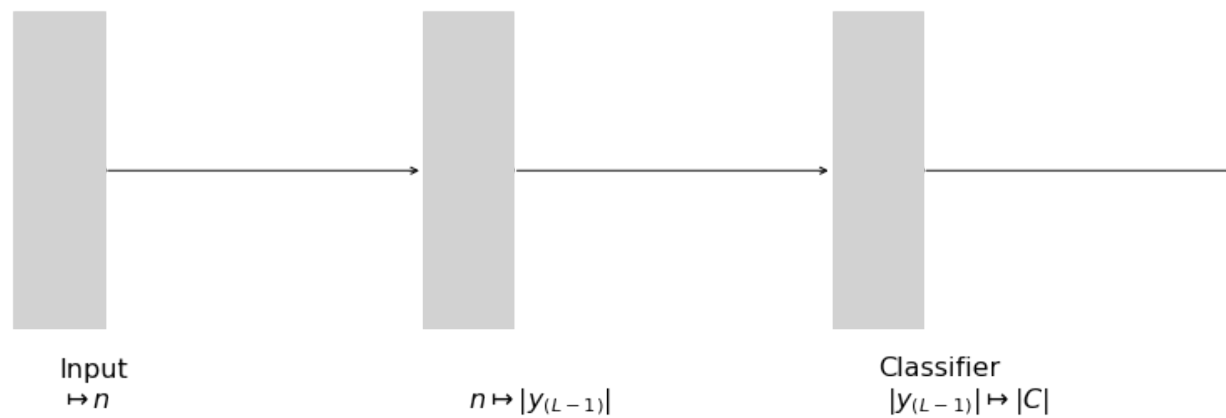
- initial layer 0 is Input layer:
 - outputs the network's inputs $\mathbf{x}$

$$\mathbf{y}_{(0)} = \mathbf{x}$$
- final layer L transforms its input $\mathbf{y}_{(L-1)}$ to prediction $\hat{\mathbf{y}}$

$$\hat{\mathbf{y}} = \mathbf{y}_{(L)}$$
 - transformation of layer L usually: Regression or Classification

In [7]: fig_tf_test

Out[7]:



In the above diagram

- the central box represents a sequence of 1 or more layers
- a sub-network
- just for brevity

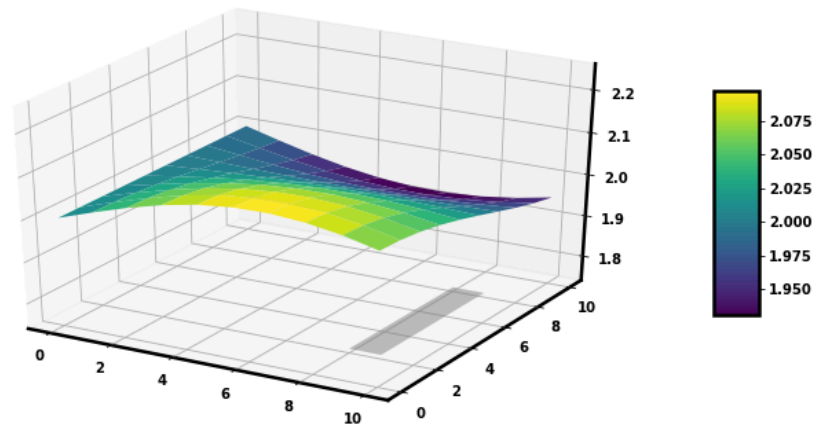
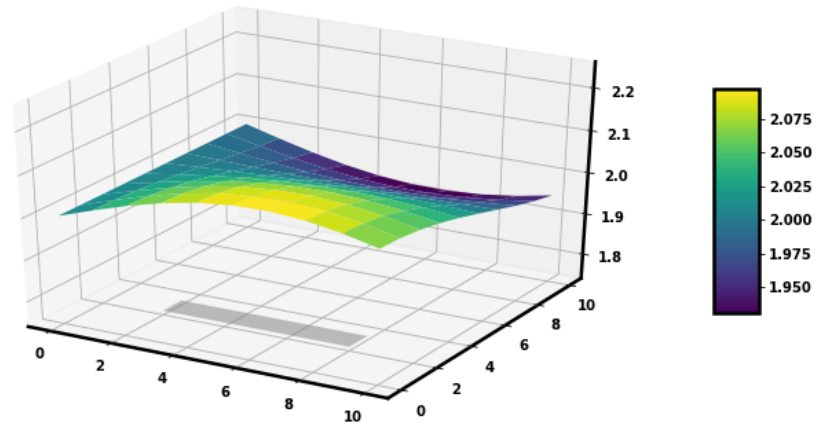
The Neural network thus computes a *function* from $\mathbf{x}$ to $\hat{\mathbf{y}}$.

The function *mimics* the training data

$$\langle \mathbf{X}, \mathbf{y} \rangle = [\mathbf{x}^{ip}, \mathbf{y}^{ip} | 1 \leq i \leq m]$$

where each *example* $\langle \mathbf{x}^{ip}, \mathbf{y}^{ip} \rangle$

- describes the mapping of the function on input $\mathbf{x}^{ip}$ to output $\mathbf{y}^{ip}$
- e.g., from input features $\mathbf{x}^{ip}$ to
 - continuous value $\mathbf{y}^{ip}$
 - discrete class $\mathbf{y}^{ip}$
 - really: output is a probability vector over finite set $\mathcal{C}$ of discrete classes



The Neural Network is trained ("learns") to mimic the training data

- by solving for the weights $\mathbf{W}_{lp}$ of each layer $1 \leq l \leq L$
- that minimize a loss function

$$\text{loss} = \sum_{i=1}^m \text{loss}^{ip}$$

- where loss^{ip} is a function of
 - how much prediction $\hat{\mathbf{y}}^{ip}$ deviates from true target/label $\mathbf{y}^{ip}$

The minimization procedure is usually a variant of *Gradient Descent*

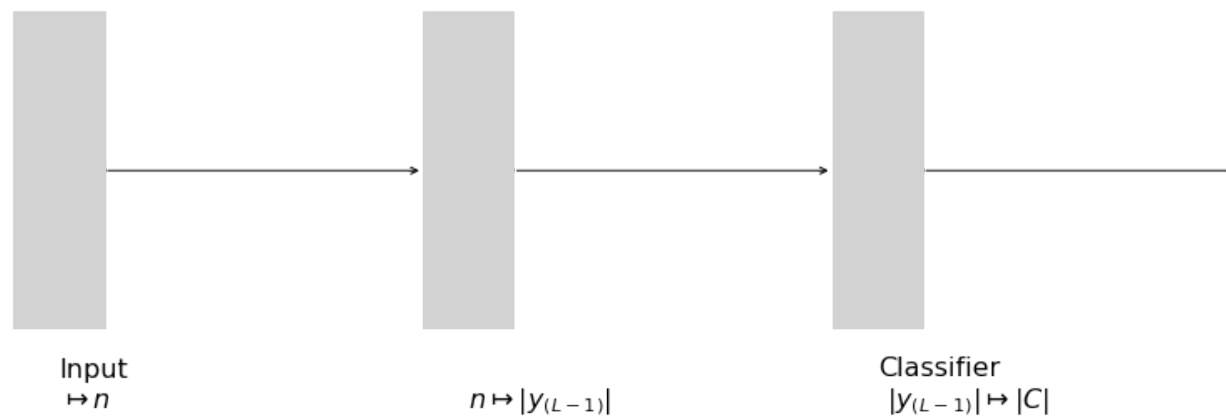
The challenge is that the operation of each layer $1 \leq l < L$

- is usually not *interpretable*
- we can describe *how* it transforms $\mathbf{y}_{(l-1)}$ to $\mathbf{y}_l$
- but not *why* it is performing the transformation
 - objective (describe) rather than subjection (why)

So the sub-network in the diagram

In [8]: fig_tf_test

Out[8]:



computes an *unknown* function

- from input of length n (the raw input)
- to a vector of length $|\mathbf{y}_{(L-1)}|$

whose purpose is

- to make the final layer L (e.g., the Classifier)
- create predictions (outputs)
- that have a very-low loss loss
 - a good "approximation" of the function described by the training data

In [9]: `print("Done")`

Done

